

Flight Delay Prediction – AI Capabilities Documentation

1. Overview

This project implements an **end-to-end intelligent flight delay prediction system** designed for a travel application.

The system predicts the probability of a flight arriving late (≥ 15 minutes) using historical flight data and provides **actionable, user-friendly insights** through both a graphical interface and a conversational AI assistant.

The solution combines:

- A **calibrated machine learning model**
- A **robust feature engineering pipeline**
- A **FastAPI-based prediction service**
- A **Streamlit-based intelligent user interface**
- Optional **OpenAI-powered conversational capabilities**

The architecture follows modern ML engineering best practices, emphasizing **probability reliability, explainability, and production readiness**.

2. High-Level Architecture

User |— Streamlit UI (Prediction + Chat) | |— Flight input form | |— Visual risk display (LOW / MODERATE / HIGH) | |— Conversational AI assistant | |— FastAPI Backend (Prediction Service) |— Feature Builder |— CatBoost ML Model |— Probability Calibration (Platt Scaling) |— Explanation Engine |— Logging & Monitoring

Key Design Principle

The UI never performs ML logic.

All predictions, calibration, and explanations are handled by the backend, ensuring correctness and consistency.

3. Machine Learning Pipeline

3.1 Model Choice

- **CatBoostClassifier**
 - Handles categorical features natively
 - Strong performance on tabular, mixed-type data
 - Robust to feature interactions common in airline data

3.2 Target Definition

- **IS_DELAYED = 1** if arrival delay ≥ 15 minutes
- Binary classification problem with **class imbalance**

3.3 Feature Engineering

Key feature groups:

- **Temporal**
 - Month, day of week, weekend flag
 - Departure time bins
 - Seasonal indicators
- **Route & Distance**
 - Distance, distance category, air time
 - Short / medium / long haul flags
- **Operational Context**
 - Hub airports
 - Busy airports
 - Hub-to-hub routes
- **Historical Risk Encodings**
 - Origin risk
 - Destination risk
 - Carrier risk
 - Route risk
 - Carrier-route risk
- **Train-only Count Features**
 - Airport traffic counts
 - Route frequency counts

All risk encodings and counts are computed **only on training data**, ensuring no data leakage.

4. Probability Calibration (Critical AI Capability)

Problem

Tree-based models (including CatBoost) often produce **overconfident probabilities**, which are misleading in user-facing applications.

Solution: Platt Scaling (Sigmoid Calibration)

- A **logistic regression calibrator** is trained on a holdout calibration split
- Raw model probabilities are converted to log-odds
- The calibrated probability is returned to the user

Result

- Probabilities become **realistic and well-spread**
- Risk labels align with true observed delay rates
- Enables trustworthy UX like:

"This flight has a 27% delay risk"

5. Backend Prediction Service (FastAPI)

Core Responsibilities

1. Validate incoming requests
2. Build features **exactly matching training**
3. Run model inference
4. Apply probability calibration
5. Assign risk category
6. Generate explanations
7. Log request and prediction metadata

API Endpoints

- GET /health
- POST /predict
- POST /recommend/times
- POST /recommend/airlines

Risk Bands (Product-Aligned)

Risk Level	Probability
LOW	< 25%
MODERATE	25% – 50%
HIGH	≥ 50%

6. Explainability & User Insights

Instead of exposing raw SHAP values, the system provides **human-readable explanations**, such as:

- "Origin JFK has elevated delay risk"
- "Evening departures tend to have cascading delays"
- "Carrier DL has strong on-time performance"

This approach balances **interpretability with usability**, making insights understandable to non-technical users.

7. Conversational AI Capabilities

An optional **OpenAI-powered assistant** enables natural language interaction.

Capabilities

- Extracts flight details from free-form text
- Resolves city names to airport codes
- Converts time phrases ("evening", "5 PM") to model inputs
- Calls backend prediction or recommendation endpoints
- Summarizes results in plain language

Example

"Is Delta better than United from JFK to LAX on Friday evening?"

The assistant automatically:

1. Parses intent
2. Calls airline comparison endpoint
3. Explains the results conversationally

8. Logging & Monitoring

Logging Strategy

- Rotating log files
- Separate logs for backend and UI
- Captures:
 - Requests
 - Prediction probabilities
 - Errors and edge cases
 - Model/calibration loading status

This supports debugging, auditing, and future monitoring integration.

9. Limitations

Despite its robustness, the system has known limitations:

1. **No real-time data**
 - Does not use live weather, ATC, or airport congestion feeds
2. **Historical bias**
 - Predictions reflect historical patterns (2017–2018 data)
3. **Static model**
 - No online learning or automatic retraining

4. Distance approximation

- When distance is not provided, averages are used

5. Heuristic explanations

- Explanations are rule-based, not full causal attributions
-

10. Future Improvements

Model Enhancements

- Add live weather and METAR data
- Retrain with newer flight datasets
- Explore ensemble models
- Add SHAP-based explanation API

Product Improvements

- User-specific risk tolerance
- Personalized recommendations
- Confidence intervals for predictions

Engineering Improvements

- Scheduled retraining pipelines
 - Model monitoring & drift detection
 - API authentication & rate limiting
 - Cloud-native observability (Prometheus, Grafana)
-

11. Summary

This project demonstrates a **production-ready AI capability**, integrating machine learning, probability calibration, explainability, APIs, and conversational AI into a cohesive system.

It showcases not just prediction accuracy, but **responsible AI design**, emphasizing:

- Trustworthy probabilities
 - Clear user communication
 - Scalable architecture
-

Built with:

CatBoost · Scikit-learn · FastAPI · Streamlit · OpenAI · Python